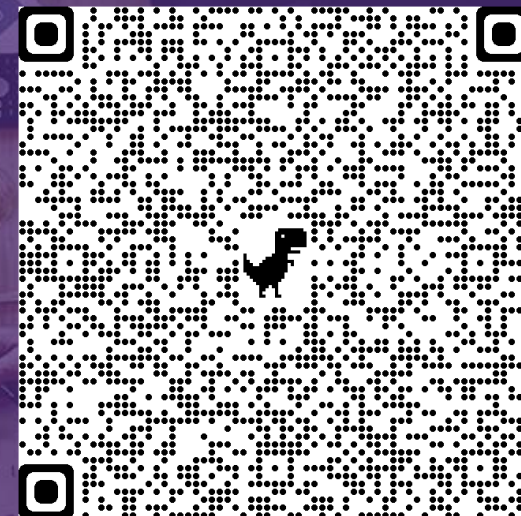


DAITAN IS NOW
encora 

Puppet Debugger

Campinas, 06Mai22



CONFIDENTIAL AND PROPRIETARY

© Encora 2022. Any use of this material without specific permission is strictly prohibited

Puppet Debugger

About myself



Roberto Nogueira

Software/Telecommunications Engineer

São José dos Campos, São Paulo, Brazil · 500+ connections

Join to follow



InMov



Universidade de São Paulo



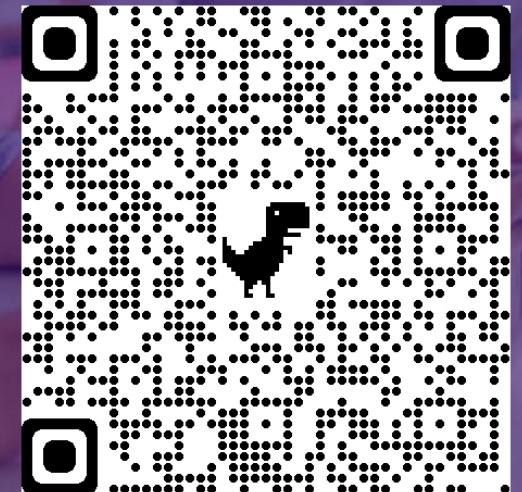
Blog

Systems Specialist at Daitan (now Encora) since 2021. Graduated in Software Engineering and Telecommunications, with more than 25 years of experience in the area and Experienced Integrator certified by **Ericsson**

Email : roberto.nogueira@encora.com

LinkedIn: https://www.linkedin.com/public-profile/settings?trk=d_flagship3_profile_self_view_public_profile

DAITAN IS NOW
encora 



Puppet Debugger

Contents

1. Objective
2. Puppet
3. Puppet Language Basics
4. Puppet Debugger
5. Puppet Debugger Commands
6. Puppet Extensions
7. References

Puppet Debugger

Objective

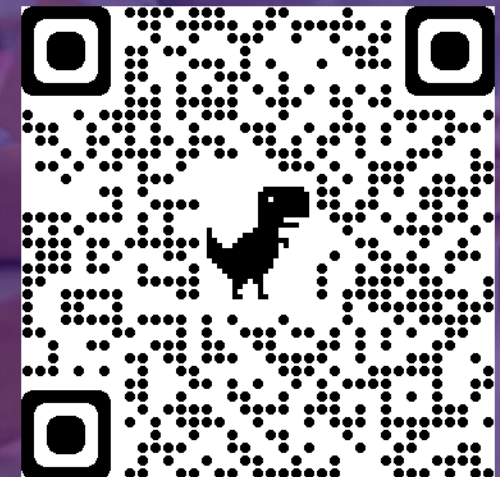
It is to introduce a [Puppet Debugger](#), a command line tool to help you write, understand, and debug Puppet code. In order to achieve that the Puppet and its language is revised.

PuppetDebugger



A command line tool to help you write, understand, and debug puppet code.

[Demo](#) is an Online Puppet Debugger.



“

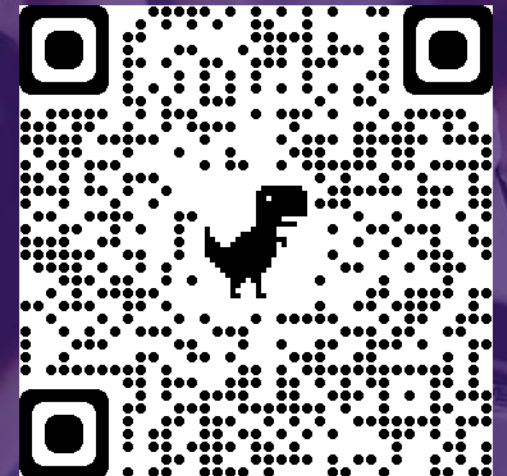
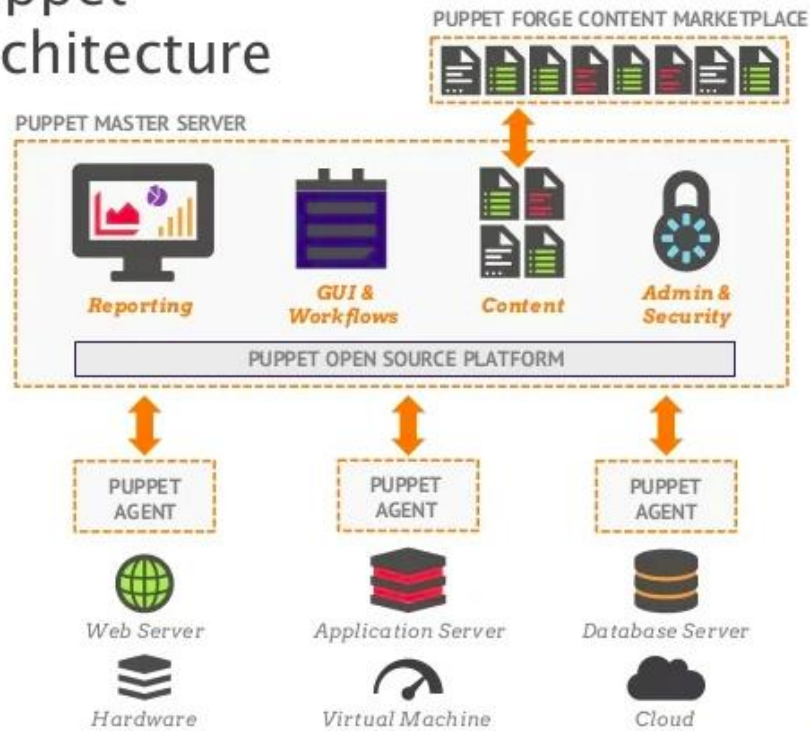
A command line tool to help you
write, understand, and debug
Puppet code

Puppet

Architecture

Puppet is a software configuration management tool which includes its own declarative language to describe system configuration.

Puppet Architecture



Puppet

Installation

Install Puppet and Puppet Debugger

```
$ docker run --name=debian11 -t -i --rm debian:11 bash  
  
$ update  
$ apt install systemctl wget lsb-release -y  
$ wget https://apt.puppetlabs.com/puppet7-release-bionic.deb  
$ dpkg -i puppet7-release-bionic.deb  
$ apt update  
$ apt-get install puppet-agent -y  
$ echo 'export PATH=$PATH:/opt/puppetlabs/puppet/bin' >> ~/.bashrc  
$ source ~/.bashrc  
$ gem install --no-document puppet-debugger
```

Puppet relevant commands

```
# print puppet configuration, usually has to be execute with sudo  
$ puppet config print  
# set log level to debug, usually has to be execute with sudo  
$ puppet config set log_level debug  
  
$ puppet --help
```

In order to install **Puppet** the way it is described here [Docker](#) is required see [Ref.1](#) for further installation.

Puppet Debugger

Puppet Language Basics

- Introduction
- Resources
- Classes
- Modules
- Variables
- Metaparameters

Puppet Language Basics

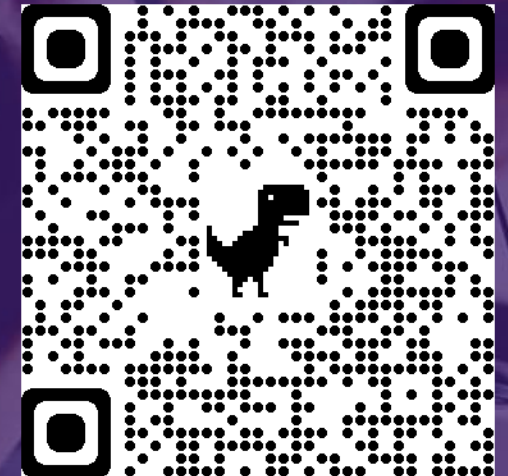
Introduction

Puppet consists of a custom [declarative language](#) to describe system configuration, which can be either applied directly on the system, or compiled into a catalog and distributed to the target system via [client-server paradigm](#) (using a [REST API](#)), and the agent uses system specific **providers** to enforce the resource specified in the manifests. The resource abstraction layer enables administrators to describe the configuration in high-level terms, such as users, services and packages without the need to specify OS specific commands (such as [rpm](#), [yum](#), [apt](#)).

DESCRIPTION



IMPLEMENTATION



Puppet Language Basics

Resources

Are the fundamental **building blocks** of the Puppet Language.

Is characteristic of a **node** that needs to be managed overtime.



Resources Types are e.g. User, Package and Service.

Puppet Language Basics

Resources

Resource Declaration adds a resource to the catalog and tells Puppet to manage that resource state.

```
resource_type { 'name':  
  attribute => value,  
}
```

```
file { '/etc/motd':  
  ensure   => 'file',  
  content  => 'Hello World',  
  owner    => 'root',  
  group    => 'root',  
  mode     => '0644',  
}
```


Puppet Language Basics

Resources

Applying the manifest

```
$ cat << 'EOF' > motd.pp
file { ['/etc/motd': ensure => 'file',
  content => 'Hello World',
  owner   => 'root',
  group   => 'root',
  mode    => '0644', }
EOF

$ ls -la motd.pp
-rw-r--r-- 1 root root 144 May 25 09:04 motd.pp

$ puppet parser validate motd.pp

$ puppet apply motd.pp
Notice: Compiled catalog for c5cd51c14f8a.br in environment production in 0.01 seconds
Notice: /Stage[main]/Main/File[/etc/motd]/content: content changed
'{sha256}a378977155fb42bb006496321cbe31f74cbda803c3f6ca590f30e76d1afad921' to
'{sha256}a591a6d40bf420404a011733cfb7b190d62c65bf0bcda32b57b277d9ad9f146e'
Notice: Applied catalog in 0.01 seconds root@c5cd51c14f8a

$ ls -la /etc/motd
-rw-r--r-- 1 root root 11 May 25 09:04 /etc/motd

$ cat /etc/motd
Hello World
```

Puppet Language Basics

Resources Examples in Debian

Check the existent resources types

```
$ puppet resource --typesadd  
:  
file  
package  
:
```

Check the existent resources **file** type

```
$ puppet resource file '/etc/hosts'  
:  
file { '/etc/hosts':  
  ensure => 'file',  
  content =>  
'{sha256}674a2d0859a6ed81ce1e3513f6ee1f710b47c3b97301b284124d5b95cf566  
b47',  
  ctime  => '2022-05-21 12:17:40 +0000',  
  group  => 0,  
  mode   => '0644',  
  mtime  => '2022-05-21 12:17:40 +0000',  
  owner  => 0,  
  provider => 'posix',  
  type   => 'file',  
}
```

Puppet Language Basics

Resources Examples in Debian

Check the existent resource **service** type

```
$ service --status-all
[ ? ] hwclock.sh
[ - ] procps

$ puppet resource service 'procps'
service { 'procps':
  ensure => 'running',
  enable => 'false',
  provider => 'debian',
}
```

Installing a resources **package** type

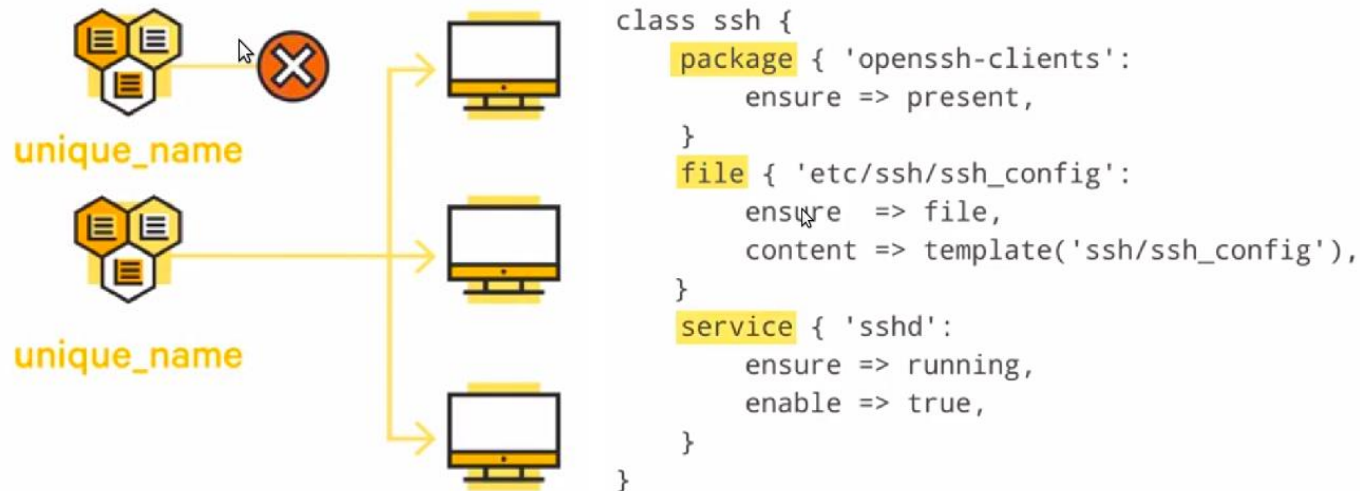
```
$ puppet resource package 'tree' ensure=present
Notice: /Package[tree]/ensure: created
package { 'tree':
  ensure => 'purged',
  provider => 'apt',
}
```

See also in [Resources](#) of [Puppet Language Basics](#) in Puppet channel in YouTube and homepage.

Puppet Language Basics

Classes

Multiple Puppet Resources can be collected and distributed together using **Classes**, by combining Resources into a Class. A Class can configure a [large block of functionality](#).

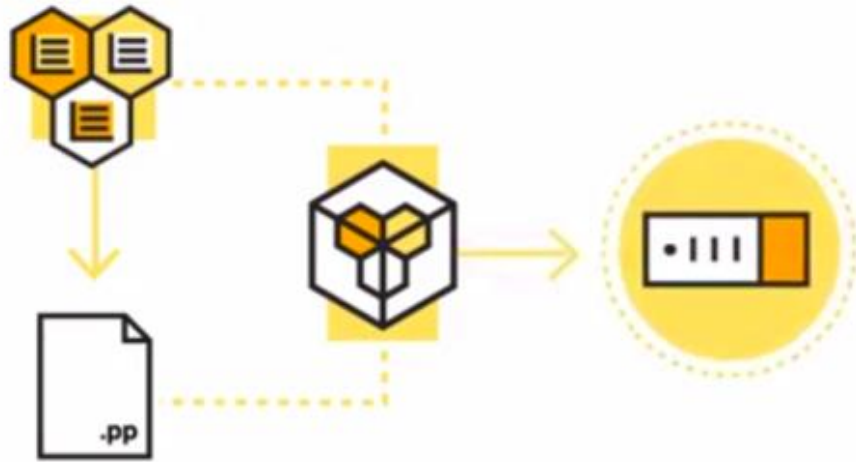


See also in [Classes](#) of [Puppet Language Basics](#) in Puppet channel in YouTube and homepage.

Puppet Language Basics

Modules

Modules are one of the most basic building blocks of the Puppet Language. They are used to manage a specific task or an application such as installing or configuring a piece of software.



Puppet Language Basics

Modules

Module is simply a directory tree that contains the files needed for a given configuration. This directory structure allows nodes to discover and loads Classes, Resources Types and more.

```
module-name/  
|- manifests  
    |- init.pp  
|- files  
|- templates  
|- examples  
|- spec  
|- metadata.json
```

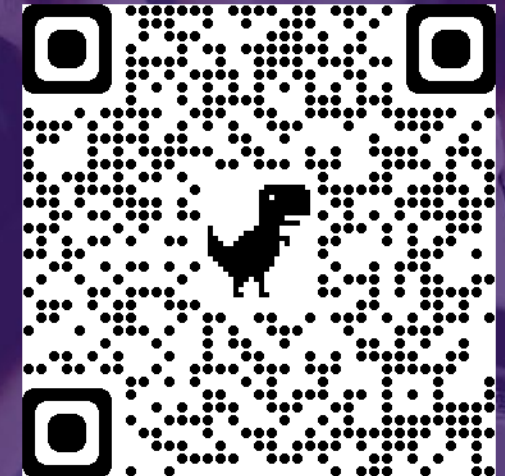
module-name is the name of the top's directory tree. If a Class is defined in a Module you can declare that name in any Manifest. When Puppet Master compiles the catalog it will dynamic load the Manifest that contain the Class definition.

Puppet Language Basics

Modules

There are thousand of Modules available for downloading at [Puppet Forge](#), written by Puppet developers and community members. There are compatible with multi-platforms.

The screenshot displays the Puppet Forge website interface. At the top, there's a navigation bar with links for Home, Docs, Forge, Learn, Contact, and Pipelines Login. Below this, a search bar is present with filters for 'What do you want to automate?', 'Supported/Approved' (Any), 'Operating System' (Any), and 'With Tasks?' (Any). The main content area is divided into three columns: 'Top Supported modules' featuring 'stdlib' and 'panos' by puppetlabs; 'Partner modules' featuring 'ovm_config' and 'wis_config' by enterprisemodules; and 'New' with a section for 'Automate simple and complex tasks' describing Puppet Bolt. Below these, there's a 'New to modules?' section with links to the 'Puppet Development Kit' and 'Resources'. On the right side of this section, there are three boxes labeled 'SUPPORTED', 'PARTNER', and 'APPROVED' with descriptions of their respective module categories.



Puppet Language Basics

Modules

Puppet Module relevant commands

```
# list puppet modules and its path, usually has to be execute with sudo
$ puppet module list
/home/rnogueira/.puppetlabs/etc/code/modules
├── puppet-nodejs (v9.0.1)
├── puppetlabs-apt (v8.3.0)
└── puppetlabs-stdlib (v4.25.1)
/opt/puppetlabs/puppet/modules (no modules installed)

$ puppet config print modulepath
modulepath/home/rnogueira/.puppetlabs/etc/code/modules:/opt/puppetlabs/puppet
/modules

# install puppet modules, usually has to be execute with sudo
$ puppet module install stdlib
:
```

See also in [Modules](#) of [Puppet Language Basics](#) in Puppet channel in YouTube and homepage.

Puppet Language Basics

Variables

Variables exist in order to be interpolated in strings to avoid rework in case the value is updated.

```
file { 'path/to/file/share/directory1': $file_share_path = 'new/path/to/file/share'
  ensure => directory,
}
file { 'path/to/file/share/directory2': }
file { 'path/to/file/share/directory3': }

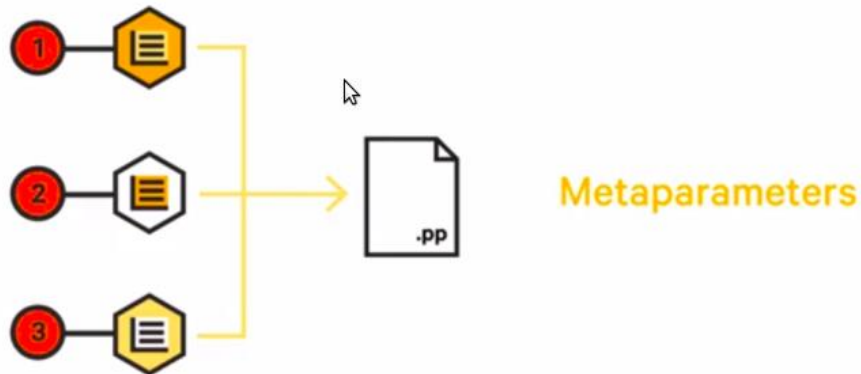
file { "${file_share_path}/directory1":
  ensure => directory,
}
file { "${file_share_path}/directory2":
  ensure => directory,
}
file { "${file_share_path}/directory3":
  ensure => directory,
}
```

See also in [Variables](#) of [Puppet Language Basics](#) in Puppet channel in YouTube and homepage.

Puppet Language Basics

Metaparameters

When a Puppet manifest is loaded the resources inside the manifest are managed the order they are declared. But when you need a group of resources to be managed in different or specific order you must explicit declare the relationships there where the **Metaparameters** come in. **Metaparameters** are special attributes that can be used in any resource type, and specify how your system act to the resource rather than mapping directly to the system state.



Puppet Language Basics

Metaparameters

The Puppet Language uses four Metaparameters to establish relationships.

- **'before'** causes a resource to be applied before the target resource.
- **'require'** causes a resource to be applied after the target resource.
- **'notify'** causes a resource to be applied *before* the target resource, but will refresh only if the notifying resource changes.
- **'subscribe'** causes a resource to be applied *after* the target resource, again refreshing only if the target resource changes.

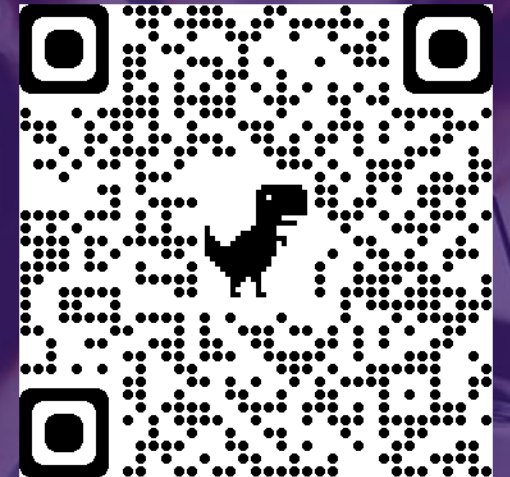
```
file { '/etc/ssh/sshd_config':  
  ensure => file,  
  mode   => 600,  
  source => 'puppet:///modules/sshd/sshd_config',  
  require => Package['openssh-server'],  
}  
  
package { 'openssh-server':  
  ensure => present,  
  before => File['/etc/ssh/sshd_config'],  
}  
  
service { 'sshd':  
  ensure   => running,  
  enable   => true,  
  subscribe => File['/etc/ssh/sshd_config'],  
}
```

See also in [Metaparameters](#) of [Puppet Language Basics](#) in Puppet channel in YouTube and homepage.

Puppet Debugger

Features

- Puppet Code and Language Evaluator
- Fully Featured **REPL**
- Mock real or fake systems using **facterdb**
- Works with puppet **4.x**, **5.x**, **6.x** and **7.x**
- Easy Gem installation
- Easy Code Playback
- Set Breakpoints
- List all functions, datatypes, types
- Plugin system for additional functionality
- Test snippets of code in realtime



Puppet Debugger

Puppet Debugger Commands

The debugger comes with many **commands** that allow you to inspect the catalog, your code or reference items currently in scope.

```
1:>> commands
Context
  environment      Show the current environment name
  reset            Reset the debugger to a clean state.
  whereami         Show code surrounding the current context.

Editing
  play             Playback a file or URL as input.

Environment
  datatypes        List all the datatypes available in the environment.
  functions         List all the functions available in the environment.
  types            List all the types available in the environment.

Help
  commands         List all available commands, aka. this screen
  exit             Quit Puppet Debugger.
  help             Show the help screen with version information.

Node
  classification    Show the classification details of the node.
  facterdb_filter   Set the facterdb filter
  facts            List all the facts associated with the node.

Scope
  classes          List all the classes current in the catalog.
  krt              List all the known resource types.
  resources        List all the resources current in the catalog.
  set              Set the a puppet debugger config
  vars             List all the variables in the current scopes.

Tools
  benchmark        Benchmark your Puppet code.
```



Puppet Debugger Commands

Resources

List all the **resources** currently in the catalog. Remember the puppet debugger is like an **interactive manifest file**. When you type puppet code, puppet will evaluate it and add it to the catalog.

```
1:>> include nodejs
2:>> resources
Resources not shown in any specific order
[
  [ 0] "Stage['main']",
  [ 1] "Class['Settings']",
  [ 2] "Class['main']",
  [ 3] "Node['_node_regexp__foo']",
  [ 4] "Notify['found foo node']",
  [ 5] "Class['Nodejs::Params']",
  [ 6] "Class['Nodejs']",
  [ 7] "Class['Nodejs::Install']",
  [ 8] "Package['nodejs']",
  [ 9] "Package['nodejs-devel']",
  [10] "Package['nodejs-debuginfo']",
  [11] "Package['npm']",
  [12] "File['root_npmrc']",
  [13] "Class['Nodejs::Repo::Nodesource']",
  [14] "Class['Nodejs::Repo::Nodesource::Yum']",
  [15] "Yumrepo['nodesource']",
  [16] "Yumrepo['nodesource-source']",
  [17] "File['/etc/pki/rpm-gpg/NODESOURCE-GPG-SIGNING-KEY-EL']"
]
```

Puppet Debugger Commands

Variables

Variables in puppet code help us remove statically defined data.

```
8:>> $file_base_path = "/tmp"
=> "/tmp"
9:>> $file_path = "${file_base_path}/test.txt"
=> "/tmp/test.txt"
10:>>
```

Listing all the **variables**

```
10:>> ls
"Facts were removed for easier viewing"
{
  "datacenter"      => "datacenter1",
  "facts"           => "removed by the puppet-debugger",
  "file_base_path"  => "/tmp",
  "file_path"       => "/tmp/test.txt",
  "module_name"     => "",
  "name"            => "main",
  "server_facts"    => {
    "environment"  => "production",
    "serverip"     => nil,
    "servername"   => "MacBook-Pro-162.local",
    "serverversion" => "6.4.0"
  },
  "title"           => "main",
  "trusted"         => {
    "authenticated" => "local",
```


Puppet Debugger Commands

Ls or Vars

The `ls` or `vars` command will list all the variables currently set, in the current scope and even in the scope of an evaluated resource.

```
8:>> ls
"Facts were removed for easier viewing"
{
  "datacenter"    => "datacenter1",
  "facts"         => "removed by the puppet-debugger",
  "module_name"   => "",
  "name"          => "main",
  "server_facts"  => {
    "environment" => "production",
    "serverip"    => "10.0.3.154",
    "servername"  => "MacBook-Pro-162",
    "serverversion" => "6.4.0"
  },
  "title"         => "main",
  "trusted"       => {
    "authenticated" => "local",
    "certname"      => nil,
    "domain"        => nil,
    "extensions"    => {},
    "hostname"      => nil
  }
}
```

Puppet Debugger Commands

Classes

The **classes** command list all the class level resources found in the current catalog. A new catalog is compiled with each entry so the list will grow over as you use more classes.

```
4:>> include stdlib
=> Puppet::Type::Component {
  loglevel => notice,
  name => "Stdlib",
  title => "Class[Stdlib]"
}
5:>> classes
[
  [0] "settings",
  [1] "stdlib",
  [2] "stdlib::stages"
]
5:>>
```

Puppet Debugger Commands

Facts

The debugger will get facts from **FactorDB** to speed up the initial loading process. These facts are then used in the scope during compilation.

To list the facts in the scope simply run the **facts** command or **\$::facts**

```
6:>> facts
{
  "architecture"      => "x86_64",
  "augeas"             => {
    "version" => "1.4.0"
  },
  "augeasversion"      => "1.4.0",
  "bios_release_date"  => "12/01/2006",
  "bios_vendor"        => "innotek GmbH",
}

6:>> $facts
{
  "architecture"      => "x86_64",
  "augeas"             => {
    "version" => "1.4.0"
  },
  "augeasversion"      => "1.4.0",
  "bios_release_date"  => "12/01/2006",
  "bios_vendor"        => "innotek GmbH",
}
```

You can force the debugger to use real facts from the current node by disabling **FactorDB**:

```
puppet debugger --no-facterdb --test -e '$os[family]'
```

Puppet Debugger Commands

FactorDB

The **puppet-debugger** internally leverages the [facterdb](#) gem to load pre-cached facts into the debugger session. You can tell the debugger to use a different set of facts from a different kernel, **OS version**, facter version or any custom combination that facterdb supports. This is all controlled via the `facterdb-filter`.

```
puppet debugger --facterdb-filter 'facterversion=/^3.6./ and operatingsystem=windows' --test -e '$os[family]'
```

```
puppet debugger --facterdb-filter 'facterversion=/^3.6./ and osfamily=RedHat' --test -e '$os[family]'
```

```
puppet debugger --facterdb-filter 'facterversion=/^3.6./ and osfamily=Darwin' --test -e '$os[family]'
```

```
{  
  "name": "macbook-pro-10.domain",  
  "values": {  
    "custom_datacenter_fact": "iceland",  
    "puppetversion": "5.3.2",  
    "architecture": "x86_64",  
    "kernel": "Darwin",  
    "domain": "domain"  
  }  
}
```

You can force the debugger to use real facts from the **current node** too:

```
puppet debugger --no-facterdb --test -e '$os[family]'
```


Puppet Debugger Commands

Types

The **types** command is used to list all the discoverable native resource types you can use in your puppet code.

```
11:>> types
[
  [ 0] "stage",
  [ 1] "whit",
  [ 2] "schedule",
  [ 3] "tidy",
  [ 4] "resources",
  [ 5] "group",
  [ 6] "file",
  [ 7] "notify",
  [ 8] "filebucket",
  [ 9] "exec",
  [10] "package",
  [11] "service",
  [12] "user",
  [13] "file_line",
  [14] "anchor",
  [15] "zpool",
  [16] "zfs",
  [17] "selboolean",
  [18] "selmodule",
  [19] "scheduled_task",
  [20] "sshkey",
  [21] "ssh_authorized_key",
  [22] "yumrepo",
  [23] "mount",
  [24] "augeas",
  [25] "zone",
  [26] "host",
  [27] "cron"
]
```

Puppet Debugger Commands

Functions

Puppet Debugger was designed to help you determine what a function returns in real time as well as what functions are available from each module.

The debugger will search your module path for functions and present you with a nice list of functions for you to explorer and execute.

To list the functions just use the command **functions**.

```
4:>> functions
FULL_NAME      | MOD_NAME
-----|-----
abs            | puppet-6.4.0
alert          | puppet-6.4.0
all            | puppet-6.4.0
annotate       | puppet-6.4.0
any            | puppet-6.4.0
any2array      | stdlib
any2bool       | stdlib
assert_private | stdlib
assert_type    | puppet-6.4.0
base64         | stdlib
basename       | stdlib
binary_file    | puppet-6.4.0
bool2num       | stdlib
bool2str       | stdlib
```

Puppet Debugger Commands

Play (Playback Support)

Playing back **files** or **urls** simply loads the content into the debugger session.

<https://gist.githubusercontent.com/logicminds/f9b1ac65a3a440d562b0/raw>

 **controls.pp**

Raw

```
1 $var1 = 'test'
2 file{"/tmp/${var1}.txt": ensure => present, mode => '0755'}
3 vars
```

```
1:>> play https://gist.githubusercontent.com/logicminds/f9b1ac65a3a440d562b0/raw
>> $var1 = 'test'
=> "test"
>> file{"/tmp/${var1}.txt": ensure => present, mode => '0755'}
=> Puppet::Type::File {
  "checksum" => "sha256",
  "ensure" => "present",
  "links" => "manage",
  "loglevel" => "notice",
  "max_files" => 0,
  "mode" => "0755",
  "name" => "/tmp/test.txt",
  "path" => "/tmp/test.txt",
  "provider" => "posix",
  "purge" => false,
```

Puppet Debugger Commands

Play (Web Based Playback Support)

If using the [web based debugger](#) you can playback a shared url which would start a debugger session and then load the content from the url or parameter.

Examples:

Urls

<https://demo.puppet-debugger.com/play?url=https://gist.github.com/logicminds/64f0fe9f64339f18f097a9f42acd6276>
<https://demo.puppet-debugger.com/play?url=https://gist.github.com/logicminds/4f6bcfd723c92aad1f01f6a800319fa4>

Simple commands

<https://demo.puppet-debugger.com/play?content=vars>

Content Parameter

[https://demo.puppet-debugger.com/play?content=md5\('dafsdffdsfasdfds'\)](https://demo.puppet-debugger.com/play?content=md5('dafsdffdsfasdfds'))

The [demo](#) site is a web application that runs a local instance of the debugger per session.

So create a link, test it and share it.

Puppet Debugger Commands

Play (Web Based Playback Support)

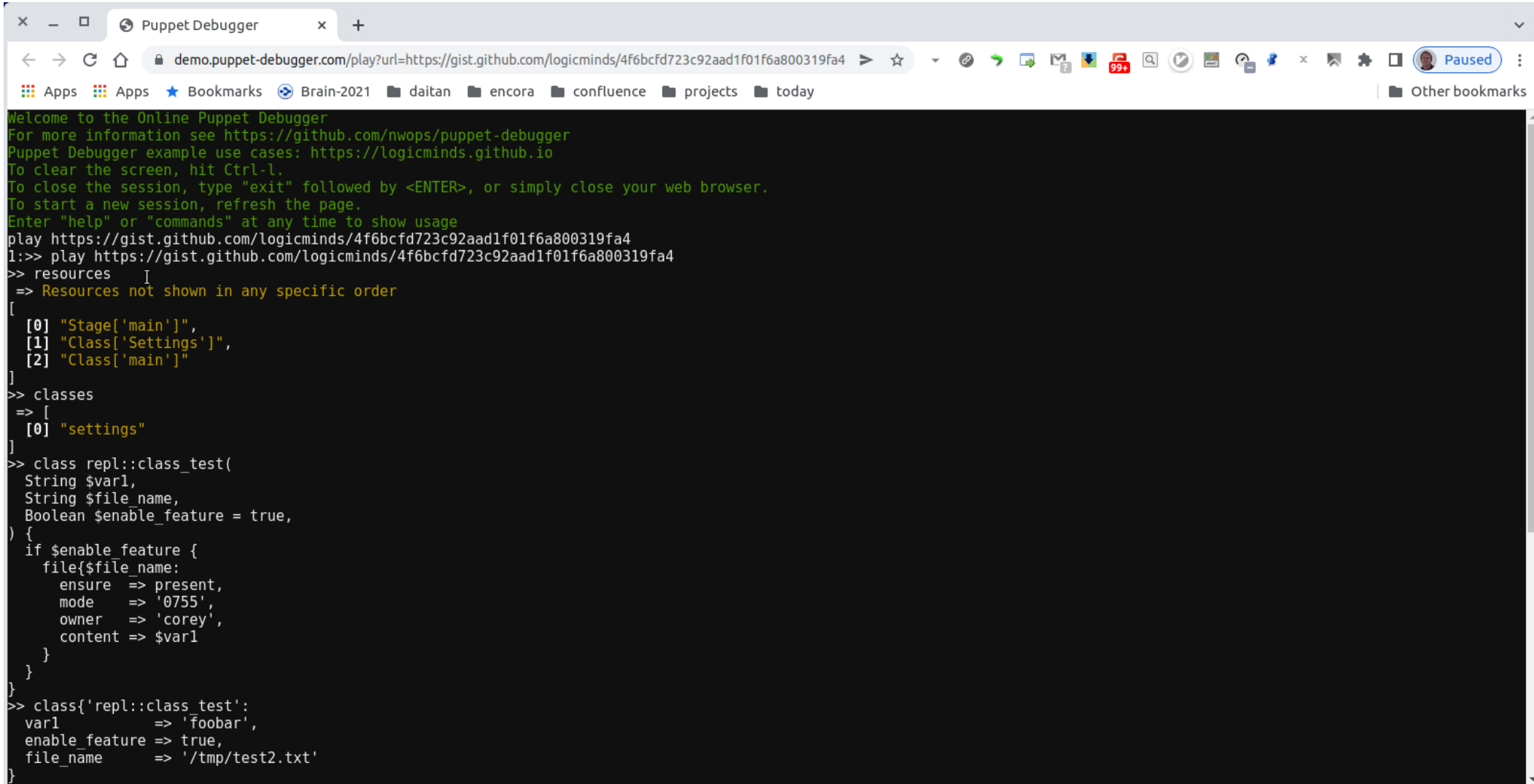
<https://gist.github.com/logicminds/4f6bcfd723c92aad1f01f6a800319fa4>

```
repl_test.pp
Raw
1 resources
2 classes
3 class repl::class_test(
4   String $var1,
5   String $file_name,
6   Boolean $enable_feature = true,
7 ) {
8   if $enable_feature {
9     file{$file_name:
10       ensure => present,
11       mode    => '0755',
12       owner   => 'corey',
13       content => $var1
14     }
15   }
16 }
17 class{'repl::class_test':
18   var1          => 'foobar',
19   enable_feature => true,
20   file_name     => '/tmp/test2.txt'
21 }
22 resources
23 classes
```

<https://demo.puppet-debugger.com/play?url=https://gist.github.com/logicminds/4f6bcfd723c92aad1f01f6a800319fa4>

Puppet Debugger Commands

Play (Web Based Playback Support)



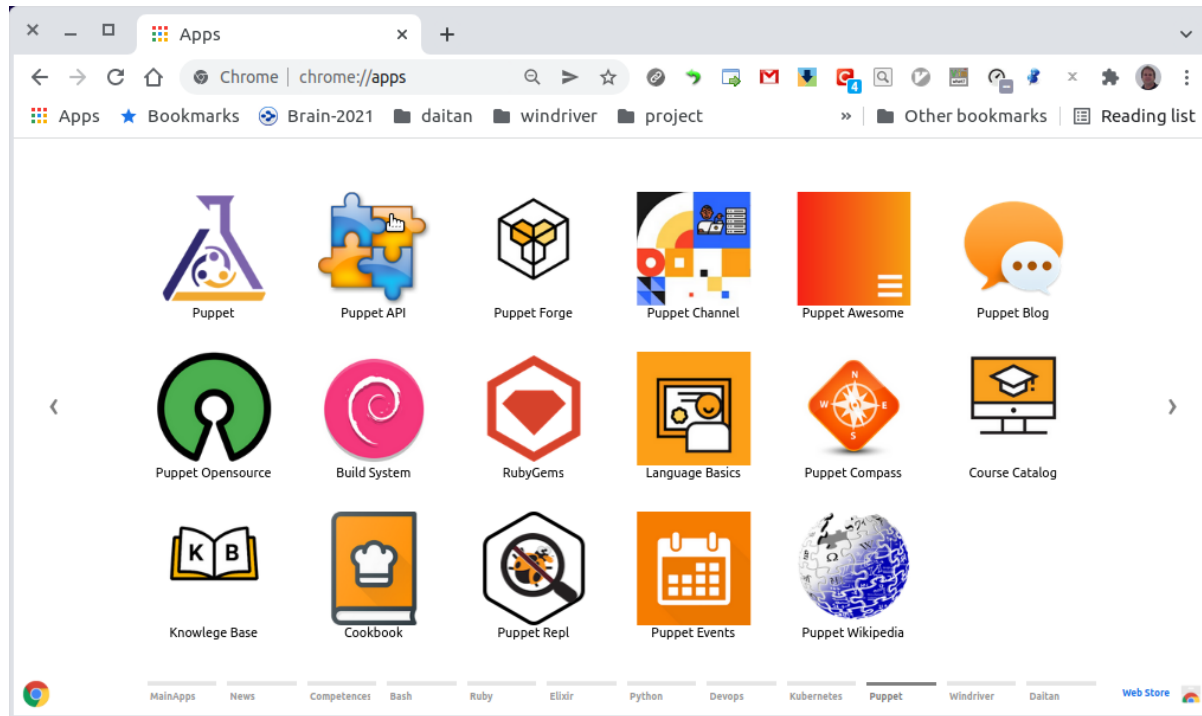
The screenshot shows a web browser window titled "Puppet Debugger". The address bar shows the URL `demo.puppet-debugger.com/play?url=https://gist.github.com/logicminds/4f6bcfd723c92aad1f01f6a800319fa4`. The browser's bookmark bar includes "Apps", "Brain-2021", "daitan", "encora", "confluence", "projects", and "today". The main content area is a terminal window with the following text:

```
Welcome to the Online Puppet Debugger
For more information see https://github.com/nwops/puppet-debugger
Puppet Debugger example use cases: https://logicminds.github.io
To clear the screen, hit Ctrl-L.
To close the session, type "exit" followed by <ENTER>, or simply close your web browser.
To start a new session, refresh the page.
Enter "help" or "commands" at any time to show usage
play https://gist.github.com/logicminds/4f6bcfd723c92aad1f01f6a800319fa4
l:>> play https://gist.github.com/logicminds/4f6bcfd723c92aad1f01f6a800319fa4
>> resources
=> Resources not shown in any specific order
[
  [0] "Stage['main']",
  [1] "Class['Settings']",
  [2] "Class['main']"
]
>> classes
=> [
  [0] "settings"
]
>> class repl::class_test(
String $var1,
String $file_name,
Boolean $enable_feature = true,
) {
  if $enable_feature {
    file{$file_name:
      ensure => present,
      mode   => '0755',
      owner  => 'corey',
      content => $var1
    }
  }
}
>> class{'repl::class_test':
var1      => 'foobar',
enable_feature => true,
file_name  => '/tmp/test2.txt'
}
```

Puppet Debugger

Puppet Extensions

[Chrome](#) and [VSCode](#) can help you stay productive and get more out of your browser and code editor.



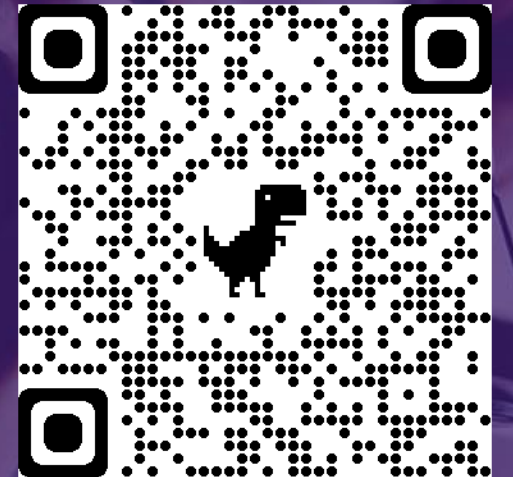
[Puppet](#) - Chrome apps for Puppet configuration management.

[Ruby](#) - Chrome apps for Ruby programming language.

For further info about Chrome Apps check [Create and publish custom Chrome apps & extensions](#).

[puppet.puppet-vscode](#) - Puppet Visual Studio Code Extension.

For further extensions check [VSCode marketplace](#).



Puppet Debugger

References

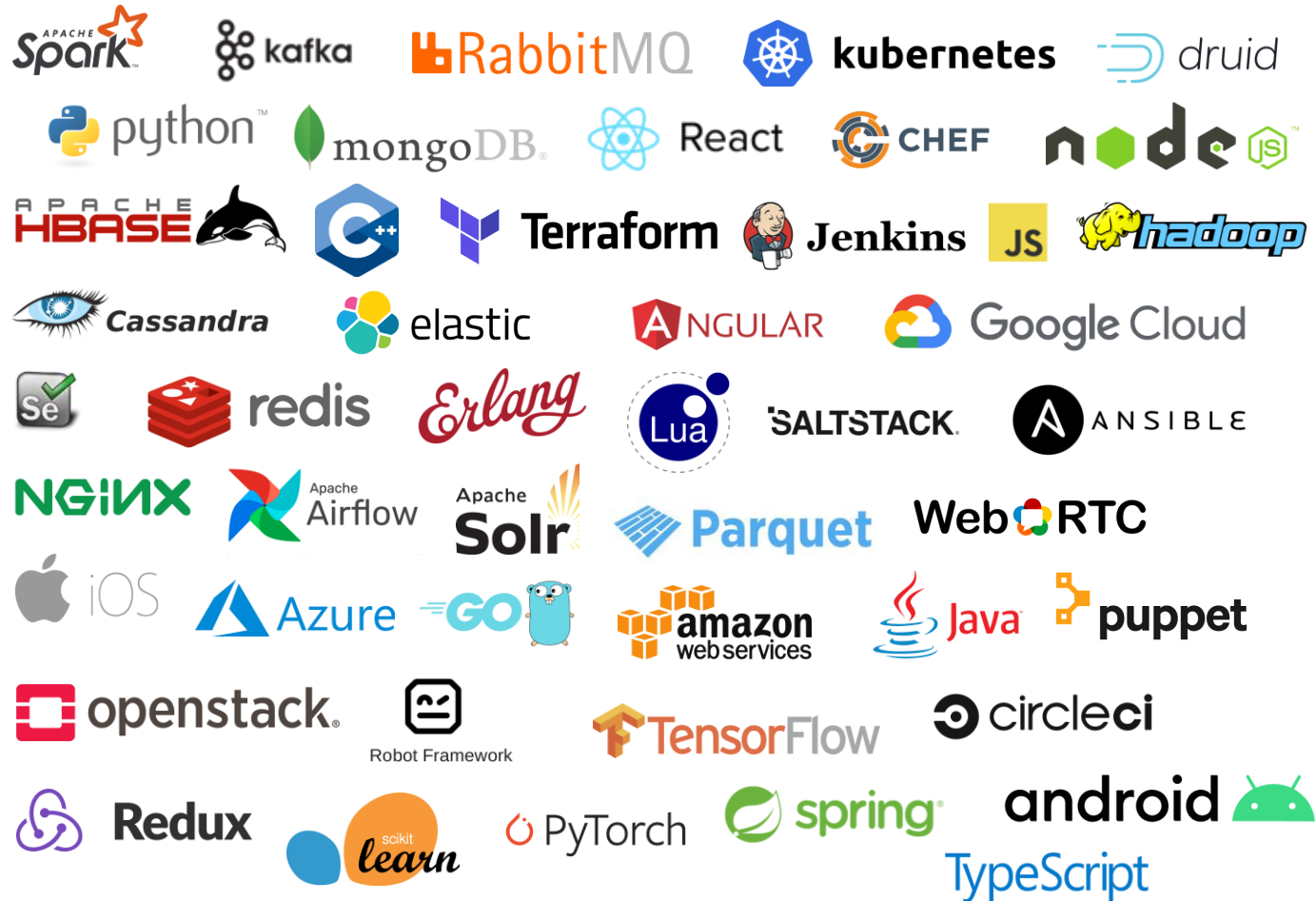


1. [Puppet in Wikipedia](#)
2. [Docker Installation](#)
3. [Puppet Language Basics](#)
4. [Puppet Forge](#)
5. [Puppet Debugger](#)
6. [Using the Puppet Debugger for Lightweight Exploration - Corey Osman](#)
7. [Puppet VS Code Tips and tricks](#)
8. [Puppet YouTube Channel](#)
9. [Puppet Cookbook](#)
0. [Awesome Puppet](#)

Accelerate with Confidence

Experience with Hundreds of tools and Platforms

DAITAN IS NOW
encora 



*Our experience with
software development
tools and technologies
accelerates deliverables
and remove risks from
schedules and budgets*

Thank you!

The quality software development partner who **significantly accelerates** your time-to-market.

DAITAN IS NOW
encora

